

Maven Lifecycle – Step-by-Step Practical Guide

Maven Lifecycle Overview

Maven follows a fixed lifecycle. Running one phase automatically runs all previous phases in order.

1. Validate

Checks project structure and pom.xml is correct.

```
mvn validate
```

2. Compile

Compiles source code into .class files.

```
mvn compile  
# Output: target/classes/
```

3. Test

Runs unit tests using JUnit/TestNG. Build fails if tests fail.

```
mvn test
```

4. Package

Creates JAR/WAR file after successful compile and test.

```
mvn package  
# Output: target/app.jar
```

5. Verify

Runs additional checks like integration tests.

```
mvn verify
```

6. Install

Installs artifact into local repository (~/.m2/repository).

```
mvn install
```

7. Deploy

Deploys artifact to remote repository (Nexus/Artifactory).

```
mvn deploy
```

Lifecycle Flow

validate → compile → test → package → verify → install → deploy

Parent POM

Used to share common configurations and dependencies across multiple projects.

```
<parent>
  <groupId>com.company</groupId>
  <artifactId>parent-project</artifactId>
  <version>1.0</version>
</parent>
```

Dependency Scope

Defines where a dependency is available in the lifecycle.

```
compile -> available everywhere
provided -> compile only, not packaged
runtime -> needed at runtime only
test -> only for testing
```

Real-Life Example (Spring Boot Project)

Typical workflow when building a project in real life.

```
# Step 1: Clean old build
mvn clean

# Step 2: Build project
mvn install

# Step 3: Run application
java -jar target/app.jar
```